

Projektplan

FollowMe

David Scherer
Philipp Gorke
Samuel Lechner

Inhaltsverzeichnis

1. Projektziele	2
2. Projektorganigramm	3
3. Work-Breakdown-Structure (WBS)	4
4. Aufwandschätzung	5
5. Ressourcen- und Meilensteinplanung	6
6. Technische Planung	6
7. GCP Budget Schätzung	7

1. Projektziele

Die Nachfrage nach (teil-)autonom fahrenden Fahrzeugen steigt stetig an. Man klassifiziert solche Fahrassistenzsysteme in vier Kategorien, abhängig von der Fortschrittlichkeit der zur Verfügung gestellten Technologie, die die Aufgaben eines menschlichen Fahrers unterstützen oder gar ersetzen soll.

Die erste Stufe (**SAE Level-1**) Assistiertes Fahren wird in beinahe allen Fahrzeugen unterstützt und beinhaltet Assistenzsysteme wie den Tempomat, Abstandsregelungsassistenten oder automatisches Spurhalten. Der Fahrer wird unterstützt, muss aber dennoch durchgehend eine vollständige Kontrolle des Fahrzeugs sicherstellen. Bereits Fahrzeuge dieser Stufe erlauben einen technisch kontrollierten Einfluss auf die Längs- und Querverführung.

Die zweite Stufe (**SAE Level-2**) Teilautomatisiertes Fahren ermöglicht beim Fahren auf einer Autobahn eine erweiterte Unterstützung durch automatisches Bremsen, Beschleunigen und Spurhalten. Ebenso sind Überholmanöver oder Einparkhilfen dieser Kategorie zuzuordnen. Allerdings muss auch hier der Fahrer stets dem Verkehr seine Aufmerksamkeit schenken und jederzeit bereit für eine Übernahme der Kontrolle über das Fahrzeug sein.

Fahrzeuge der Stufe drei (**SAE Level-3**) Hochautomatisiertes Fahren verfügen über erweiterte Sensorik wie LIDAR und Radar, die maßgeblich zur Wahrnehmung der Fahrzeugumgebung beitragen können. Wenngleich der Fahrer hier das System nach wie vor kurzfristig übernehmen können muss, kann er sich im Normalfall weitgehend anderen Tätigkeiten widmen, da das Fahrzeug sich eigenständig durch den (Fließ-)verkehr bewegen kann. Das aktuell einzige Serienfahrzeug, das diese Anforderung erfüllt ist der Mercedes EQS, der mit seinem Stauassistent mit bis zu 60 km/h das Fahrzeug durch den Verkehr steuern kann.

Auf die weiteren Stufen soll im Folgenden nicht eingegangen werden.

Das Ziel dieses Projekts ist es nun, dass Fahrzeuge, die mindestens über SAE Level-1 verfügen (**Following Vehicles**), die Fähigkeiten eines SAE Level-3 Fahrzeugs (**Leading Vehicle**) mitnutzen können, indem sie dessen Fahrspur folgen. Dafür müssen alle Fahrzeuge permanent relevante Fahrdaten (mehr Details siehe Projektangabe) an ein Backend senden. Im Falle, dass potentielle Following Vehicles nah genug an potentielle Leading Vehicles kommen, wird der **FollowMe** Modus aktiviert. Das vorausfahrende Fahrzeug sendet relevante Daten an die dahinterliegenden Fahrzeuge, sodass deren Verhalten dem eines SAE Level-3 fähigen Fahrzeugs weitgehend gleichgestellt werden kann. Der FollowMe Modus wird bei Verletzung der erforderlichen Bedingungen vom Backend ausgehend abgebrochen.

Ziel dieses Projektes ist es nicht, Fahrzeuge grundsätzlich SAE Level-3 fähig zu machen. Die Klassifizierung der Following Vehicles soll sich nicht ändern, zumal das rechtlich auch schwierig umzusetzen wäre. Weiters müsste die grundsätzlich rechtliche Lage des Projektes abgeklärt werden.

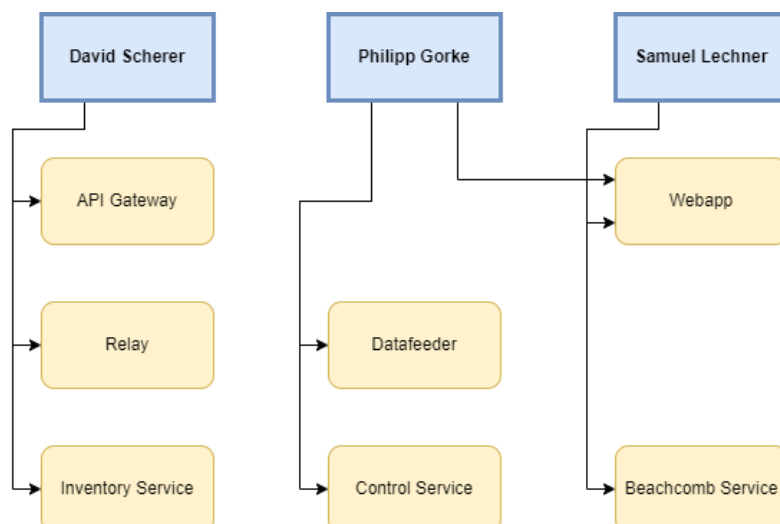
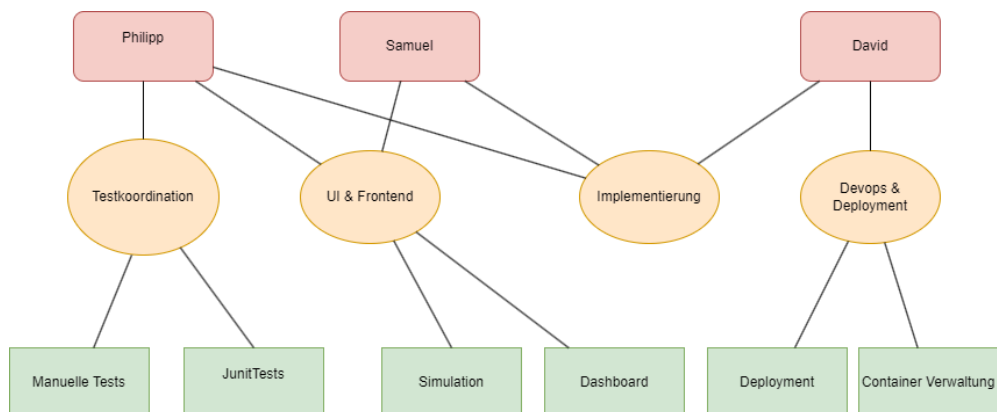
Quelle:

<https://www.adac.de/rund-ums-fahrzeug/ausstattung-technik-zubehoer/autonomes-fahren/grundlagen/autonomes-fahren-5-stufen/>

2. Projektorganigramm

David Scherer	Philipp Gorke	Samuel Lechner
Kubernetes Integration Manager (Ist Hauptbeauftragter für Deployment und Integration der Microservices und Google Cloud Platform), Dokumentationsbeauftragter	Technischer Architekt & Testkoordination (Koordination und Durchführung der Tests des Clusters, als auch Zuteilung der technischen Aufgaben innerhalb des Teames)	UI-Manager (Koordiniert die WebApp, designt diese und gibt die Entwicklung vor, zudem Organisation und Durchführung der Simulation)
Entwicklung (Api Gateway, Relay und Inventory Service)	Entwicklung (Datafeeder, Control Service und Unterstützung bei iWebapp)	Entwicklung (Webapp, Beachcomb Service)

Wir haben uns entschieden, dass innerhalb des Projektes nicht nur die Aufgaben von den Koordinatoren ausgeführt werden, sondern von allen. Die Koordinatoren sind hauptverantwortlich dafür und übernehmen die Verantwortlichen dafür.



3. Work-Breakdown-Structure (WBS)

Wir haben uns entschieden, die einzelnen Komponenten an verschiedene Personen abzugeben. In der Webapp, welche wir als größten Aufwand einschätzen, werden zwei Personen arbeiten. Dieser Ablauf ermöglicht einen parallelen Entwicklungsprozess. Zudem soll erst nach Abschluss einer Komponente mit der nächsten begonnen werden.

Als Pakete haben wir uns für Implementierung, Deployment, Testing, Durchführung der Simulation entschieden. Die Unterpakete sind wie folgt definiert.

Dokumentation:

- Projektplan [5h]
- Dokumentation der API Schnittstellen und Source-Code [8h]
- Wartungshandbuch [2h]
- Architektur & Designdokument [2h]
- Dokument: Conclusio und Lessons Learned [1h]
- Präsentationsvorbereitung [2h]

Implementierung:

- Passway [10h]
- WebApp [15h]
- Inventory Service [5h]
- Beachcomb Service [5h]
- Control Service [10h]
- Relay [10h]
- Datafeeder [5h]

DevOps:

- Deployment auf Kubernetes [5h]
- Dockerizing [5h]
- Google Services wie z.B. Datenbank verwenden [5h]
- Messaging [10h]

Testing:

- Die zwei komplexesten Microservices werden als Modul Tests getestet [5h]
 - Dafür werden voraussichtlich JUnit Tests verwendet mit der Hilfe von Mockmvc um API calls zu mocken
- Zudem werden manuelle Tests mittels Postman durchgeführt [5h]

Simulation:

- Der Datafeeder soll entsprechende Daten erhalten, um eine erfolgreiche Simulation zu ermöglichen [8h]
- Die Simulation soll in einem echten Szenario durchgetestet werden und eine Screenrecording erstellt werden. [2h]

4. Aufwandschätzung

Gemäß der oben definierten Work-Breakdown-Structure soll im Folgenden eine Auflistung des geschätzten Aufwands für die jeweiligen Pakete folgen. Das genaue Zustandekommen durch Summation der Unterpakete kann im Abschnitt WBS eingesehen werden.

- Dokumentation: 20 Personenstunden
- Implementierung: 40 Personenstunden
- DevOps: 25 Personenstunden
- Testing: 10 Personenstunden
- Simulation: 10 Personenstunden
- **Insgesamt: 105 Personenstunden**

5. Ressourcen- und Meilensteinplanung

Es werden die folgenden Meilensteine angestrebt.

- M1 - Projektplan erarbeiten und formulieren - erstes Treffen, Rollenverteilung und generelles Konzept über Implementierung und Projekt erarbeiten
- M2 - Architektur- und Designdokument soll formuliert werden. Zudem sollen parallel die Schnittstellen der einzelnen Microservices definiert werden und die Implementierung des Passways starten. Außerdem sollte ein Gitprojekt angelegt sein
- M3 - Die Implementierung des Passways sollte finalisiert werden. Zudem beginnt die Implementierung der Service Microservices. Beginn der Entwicklung der Webapp.
- M4 - Implementierung aller Microservices abgeschlossen und lauffähig in den einzelnen Container. Ebenso sollten die Tests definiert sein. Webapp sollte bereits zu 70% implementiert sein.
- M5 - Implementierung sollte hier beendet sein. Zudem soll die gesamte Applikation ausreichend getestet werden. Die gesamte Applikation muss bereits deployed laufen. Die Simulation wird in diesem Meilenstein durchgeführt und entsprechend vorbereitet.
- M6 - Wartungshandbuch wird geschrieben. Simulation und alle anderen Tätigkeiten sollten bereits durchgeführt werden.

	KW 12	KW 13	KW 14	KW 15	KW 16	KW 17	KW 18	KW 19	KW 20	KW 21	KW 22	KW 23
M1												
M2												
M3												
M4												
M5												
M6												

6. Technische Planung

Wir werden die nachfolgenden Technologien verwenden. Diese wurden besonders deshalb ausgewählt, weil sie den Anforderungen des Projekts sehr gut entsprechen und dadurch

- MongoDB - wird besonders deshalb verwendet um Geo Spatial Queries zu verwenden und Daten zu speichern, welche besonders in den Distanzberechnungen in den Microservices verwendet werden
- Spring Boot - wird verwendet, um die Microservices zu implementieren und eine API Schnittstelle zur Verfügung stellen.
- REST - um das API Gateway zu implementieren und die verschiedenen Interfaces zu exposen und anzusprechen
- Docker und Kubernetes (Google Kubernetes Engine) - um die Applikation zu containerisieren und zu orchestrieren
- Angular - für das Frontend
- RabbitMQ - für den Message Broker
- Gitlab (git) als Source Code Management Tool
- Kanban: Gitlabs Issue Board
- Unit-Tests (JUnit, Python unittest) und Integration-Tests (Mockmvc), um zu testen, wie die verschiedenen Microservices zusammenspielen und ob die funktionieren

7. GCP Budget Schätzung

Von der GCP werden wir nur die Google Kubernetes Engine (GKE) brauchen, weswegen sich die Budget Schätzung nur darauf beschränken wird.

Für eine bestmögliche Schätzung verwenden wir den Pricing Calculator von der GCP (<https://cloud.google.com/products/calculator/>). Dieser inkludiert allerdings nicht den Free Tier in der Rechnung. (Additionally, GKE applies a monthly free tier worth \$74.40 for Zonal and Autopilot clusters. This free tier is applied directly to your billing account through credits and is not reflected in this calculator.)

Da es sich um ein kleines Projekt handelt, schätzen wir, dass wir nicht besonders viel Rechenleistung und Speicherkapazität benötigen werden, der machine type n1-standard-1 sollte genügen, jedoch Wert auf Availability legen und daher den Cluster durchgehend verfügbar haben wollen. Daher verwenden wir folgende Konfiguration, die im Screenshot ersichtlich ist. Wir verwenden nur einen Node (gleichzusetzen mit einer VM) und werden das gesamte Backend in diesem deployen. Die Region wurde auf Belgien festgelegt, da andere Regionen (Frankfurt oder Zürich zB), die zwar näher wären, allerdings etwas teurer sind. Der Free Tier in der GKE inkludiert nur 74,40\$ für die Cluster Management Gebühren (0.10\$ pro Stunde), beziehungsweise 744 Cluster Stunden.

Die Rechnung ergibt daher 111,14\$ pro Monat, wovon pro Monat der Free Tier von 74,40\$ abgezogen wird. Wir nehmen an, dass wir den Cluster ungefähr 3 Monate brauchen werden, weswegen sich ein Gesamtbetrag von 110,22\$ ergibt von den verfügbaren 150\$. Dadurch haben wir auch einen gewissen Spielraum, falls wir im Laufe der Entwicklung zusätzliche Dienste oder mehr Leistung benötigen.

Total estimated cost \$111.14 / mo  Download .csv  Duplicate estimate		Cost Estimate Summary As of Mar 30, 2024 • 4:14 PM Prices in USD	
		COMPUTE	\$111.14
		GKE (Kubernetes Engine)	\$111.14
		Service type	GKE
		Number of Zonal clusters	1 \$73.00
		Boot disk type	Standard persistent disk N/A
		Boot disk size (GiB)	10 GiB \$0.00
		Node-time	730 Hours N/A
		Machine type	n1-standard-1, vCPUs: 1, RAM: 3.75 GB \$38.14
		Number of Nodes	1 N/A
		Operating System / Software	Free: Ubuntu N/A
		Kubernetes Edition	GKE Standard Edition N/A
		Provisioning model	Regular N/A
		Enable Confidential GKE Nodes	false N/A
		Add sustained use discounts	false N/A
		Add GPUs	false N/A
		Local SSD	0 N/A